C2 Evasion RL

Học tăng cường mức gói tin để né phát hiện lưu lượng C2 botnet trên Snort 2.9.20 với bộ luật ET Open

Lê Hoàng Phát

Ngày 4 tháng 10 năm 2026

Tóm tắt nội dung

Luận văn này nghiên cứu câu hỏi: *một agent học tăng cường (RL) có thể tự động biến đổi các gói tin beacon C2 thật để né một IDS chữ ký thật, trong khi luồng C2 vẫn còn hoạt động được không?* Chúng tôi xây dựng môi trường `HiddenDefenderEnv` với 14 cơ chế biến đổi gói tin, dùng `MaskablePPO` để chọn cơ chế, và chấm điểm mọi hành động bằng chính nhị phân **Snort 2.9.20** (bộ luật ET Open C2, 21 374 luật) chứ không dùng mô hình xấp xỉ khi đo kết quả chính.

Kết quả chính được đo bằng Snort thật: trên tập test held-out 40 luồng, agent đạt **40/40 né hợp lệ (100%)** — không báo động, và ngữ nghĩa C2 vẫn nguyên vẹn — trong khi **control âm tính** (luồng gốc chưa biến đổi) bị Snort báo động **40/40 (tỷ lệ né 0%)**. Đây là bằng chứng rằng né tránh là thật, không phải lỗi đo lường.

Tuy nhiên, luận văn cũng báo cáo trung thực hai phát hiện *phủ định* quan trọng: (1) với defender tĩnh này, bài toán tìm kiếm gần như suy biến — mỗi framing chỉ có một hành động tối ưu gần như cố định (`split8` cho luồng nhị phân, `split24` cho luồng HTTP), nên RL **không vượt** được các baseline đơn giản; và (2) khi kiểm tra chéo capture (cross-capture), tỷ lệ né trung bình chỉ **38,89%** (độ lệch chuẩn 43,74%), và trên cả ba capture agent **hoà** với control "corrupt-everything" — nghĩa là nó không học được gì mà hành động tầm thường "làm hỏng mọi gói" không làm được. Một capture (win13) đạt 0% vì các luồng của nó gần như không có payload để biến đổi — đây là giới hạn của *không gian hành động*, không phải kết luận về họ botnet.

Luận văn đóng gói kèm một quy trình đo lường nghiêm ngặt (một cặp (flow, cơ chế) cho mỗi pcap, control âm tính bắt buộc, trần oracle, bộ verifier 81 kiểm tra tự động) và ghi lại sáu lỗi phương pháp đã từng tạo ra kết quả sai trông rất hợp lý. Đóng góp của công trình là sự trung thực của phép đo và việc chỉ ra rằng, dưới một defender tĩnh đơn luật, RL chưa tạo thêm giá trị so với tìm kiếm tham lam.

Mục lục

1 Giới thiệu & Động lực **5**
- 1.1 Bối cảnh . . . 5
- 1.2 Phát biểu bài toán . . . 5
- 1.3 Động lực hai chiều . . . 5
- 1.4 Đóng góp . . . 6
- 1.5 Phạm vi . . . 6

2 Nghiên cứu liên quan **7**
- 2.1 Ví dụ đối kháng trong học máy . . . 7
- 2.2 Phát hiện C2 và IDS chữ ký . . . 7
- 2.3 Học tăng cường và né tránh . . . 7
- 2.4 Khoảng trống mà luận văn hướng tới . . . 7

3 Thiết kế hệ thống & Mô hình mối đe dọa **9**
- 3.1 Mô hình mối đe dọa . . . 9
  - 3.1.1 Kẻ tấn công (red team) . . . 9
  - 3.1.2 Defender (blue team) . . . 9
  - 3.1.3 Ràng buộc . . . 9
- 3.2 Môi trường `HiddenDefenderEnv` . . . 9
  - 3.2.1 Quan sát (observation) . . . 9
  - 3.2.2 Không gian hành động . . . 10
  - 3.2.3 Action masking (lọc hành động khả thi) . . . 10
  - 3.2.4 Phần thưởng (reward) . . . 11
  - 3.2.5 Định nghĩa "giữ ngữ nghĩa" . . . 11

4 Phương pháp **12**
- 4.1 Dữ liệu và phân chia tập . . . 12
  - 4.1.1 Nguồn dữ liệu . . . 12
  - 4.1.2 Quy tắc chọn và phân chia . . . 12
- 4.2 Huấn luyện . . . 12
  - 4.2.1 Thuật toán . . . 12
  - 4.2.2 Hai bộ chấm điểm . . . 13
- 4.3 Quy trình đo lường . . . 13
  - 4.3.1 Một cặp (flow, cơ chế) cho mỗi pcap . . . 13
  - 4.3.2 Control âm tính (bắt buộc) . . . 13
  - 4.3.3 Trần oracle và các baseline . . . 13
  - 4.3.4 Verifier tự động . . . 14

5 Kết quả & Đánh giá **15**
5.1 Kết quả chính (Snort thật) . . . 15
5.1.1 Control âm tính . . . 15
5.2 Số liệu huấn luyện . . . 16
5.3 Ma trận cơ chế (Snort thật, 8 luồng HTTP) . . . 16
5.4 RL có thật sự cần thiết không? (phát hiện làm phức tạp câu chuyện) . . . 16
5.5 Scale-up (Snort thật, neris-20110811, 8 vòng) . . . 17
5.6 Kiểm tra chéo capture (cross-capture) . . . 17
5.7 Kiểm định surrogate so với Snort thật . . . 17
5.8 Sweep chi phí corrupt . . . 18

6 Thảo luận **19**
6.1 Vì sao `MaskablePPO` hợp ở đây . . . 19
6.2 Chi phí và ý nghĩa . . . 19
6.3 Vì sao các con số nên được tin: những lỗi đã bắt được . . . 19
6.4 Hạn chế . . . 20
6.5 Các bất nhất cần giải quyết . . . 20
6.6 Hướng phát triển . . . 20

7 Kết luận **21**

8 Phụ lục **22**
8.1 A. Thống kê corpus . . . 22
8.2 B. Chi tiết hiện thực . . . 22
8.3 C. Bảng kết quả per-flow (40 luồng test) . . . 22
8.4 D. Control âm tính và kiểm tra ngữ nghĩa . . . 22
8.5 E. Ghi chú chỉnh sửa các con số lỗi thời . . . 23

Danh sách bảng

8.1 Kết quả per-flow trên tập test held-out (Snort thật). 24

Danh sách hình vẽ

Chương 1

Giới thiệu & Động lực

1.1 Bối cảnh

Botnet là mạng lưới các máy bị nhiễm (bot) nhận lệnh từ một máy chủ điều khiển (command-and-control, C2). Kênh C2 là điểm sống còn: cắt được nó là vô hiệu hoá cả botnet. Vì vậy các hệ thống phát hiện xâm nhập (IDS) như *Snort* [Roesch, 2023] với bộ luật chữ ký mở như Emerging Threats (ET) Open [Proofpoint, Inc., 2023] tập trung vào việc khớp các mẫu byte đặc trưng trong lưu lượng C2.

Đây là một trò chơi "mèo vờn chuột": phía tấn công liên tục tìm cách che giấu kênh C2 (mã hoá, phân mảnh, chèn nhiễu), phía phòng thủ cập nhật luật để bắt lại. Việc tạo ra các biến đổi né tránh một cách thủ công tốn nhiều công sức và đòi hỏi hiểu biết sâu về giao thức. Câu hỏi đặt ra: *có thể tự động hoá quá trình này bằng học tăng cường không?*

1.2 Phát biểu bài toán

Cho một tập luồng C2 đã được gán nhãn (lấy từ các capture malware thật), mục tiêu là huấn luyện một *agent* sao cho, với mỗi luồng, nó chọn một phép biến đổi gói tin để:

(1) Snort **không** còn báo động trên luồng đó (*né tránh*), và

(2) kênh C2 vẫn còn hoạt động được — lệnh C2 vẫn tới được endpoint (*giữ ngữ nghĩa*).

Hai điều kiện này phải cùng đúng; một phép biến đổi xoá luôn kênh C2 thì không được tính là né tránh. Ngoài ra agent phải *tối thiểu hoá chi phí* biến đổi, vì sửa nhiều gói làm tăng nguy cơ bị phát hiện bằng kênh phụ và làm hỏng độ tin cậy của C2.

1.3 Động lực hai chiều

- **Phía phòng thủ:** hiểu được cách một kẻ tấn công *thực sự* có thể tạo lưu lượng trốn tránh giúp xây dựng luật phát hiện bền vững hơn.
- **Phía học thuật:** khoảng cách giữa "chấm điểm bằng mô hình xấp xỉ" và "chấm điểm bằng IDS thật" là một hạn chế cơ bản của transfer learning trong bối cảnh đối kháng; đo lường nó một cách trung thực có giá trị riêng.

1.4 Đóng góp

C1. Đường ống đầu-cuối trên IDS thật. Một vòng lặp huấn luyện RL trong đó mọi hành động được chấm bởi nhị phân Snort 2.9.20 thật (chế độ `-real-snort`), tích hợp với pcap thật từ Stratosphere IPS [Lab and in Prague, 2013].

C2. Kết quả chính có kiểm chứng. 40/40 luồng held-out né hợp lệ dưới Snort thật, đối chiếu control âm tính 40/40 bị báo động.

C3. Kết quả phủ định được báo cáo thẳng thắn. RL không vượt baseline tham lam, và kiểm tra chéo capture cho thấy policy *hoà* với control "corrupt-everything" trên cả ba capture — nên con số né chéo capture **không** phải thành tựu của RL.

C4. Kỷ luật đo lường. Quy trình một cặp (flow, cơ chế) cho mỗi pcap, control âm tính bắt buộc, trần oracle, và bộ verifier tự động 81 kiểm tra (`controls/verify_report_claims.py`) đảm bảo mọi con số trong báo cáo đều truy được về file JSON gốc.

C5. Bài học về lỗi phương pháp. Sáu lỗi (trích xuất luồng một chiều, viết lại gói tin ngây thơ, kiểm tra ngữ nghĩa lỏng, chấm điểm theo lô, luật bị throttle, cửa sổ đọc alert quá ngắn) từng tạo ra kết quả sai trông rất hợp lý, và cách chúng bị phát hiện.

1.5 Phạm vi

Kết quả được xác thực trên **bộ luật tĩnh ET Open**, trên **cùng một họ malware** (một nhị phân, nhiều ngày). Luận văn *không* tuyên bố tổng quát hoá sang mọi họ botnet, cũng không tuyên bố chống lại IDS thích ứng. Những giới hạn này được nêu rõ ở Chương 6.

Chương 2

Nghiên cứu liên quan

2.1 Ví dụ đối kháng trong học máy

Học máy đối kháng nghiên cứu cách tạo đầu vào khiến mô hình phân loại sai trong khi vẫn giữ tính hợp lệ. Biggio and Roli [2013] đặt nền móng cho *tấn công né tránh* (evasion attack); Goodfellow et al. [2014] và Carlini and Wagner [2016] phát triển các phương pháp gradient để tối ưu nhiễu đối kháng. Papernot et al. [2015] mở rộng sang miền rời rạc (văn bản, tệp nhị phân). Điểm chung: né tránh là một *bài toán tối ưu có ràng buộc*, và trong không gian rời rạc (như gói tin) thì tìm kiếm trở nên khó hơn nhiều so với tối ưu liên tục.

Trong miền malware, Pierazzi et al. [2019] và Demetrio et al. [2020] nhấn mạnh khái niệm *problem-space* (không gian bài toán): một biến đổi phải vừa qua được bộ phân loại, vừa giữ được chức năng thực thi của mẫu. Đây chính xác là ràng buộc "giữ ngữ nghĩa C2" mà chúng tôi áp dụng.

2.2 Phát hiện C2 và IDS chữ ký

Snort [Roesch, 2023] là IDS dựa trên chữ ký phổ biến, khớp các mẫu byte và điều kiện header. Bộ luật ET Open [Proofpoint, Inc., 2023] cung cấp hàng chục nghìn luật công khai, trong đó các luật C2 thường neo vào `content`, `depth` hoặc `offset` trong payload. Vì luật là tĩnh, một biến đổi làm dịch chuyển hoặc phân mảnh các byte mục tiêu có thể khiến luật không còn khớp — đây là cơ sở kỹ thuật cho các cơ chế phân mảnh mà chúng tôi dùng. Silva et al. [2017] khảo sát né tránh IDS bằng thao tác mạng dưới góc nhìn học máy.

2.3 Học tăng cường và né tránh

MDP là khung hình thức chuẩn cho quyết định tuần tự [Puterman, 1994]; `PPO` [Schulman et al., 2017] là thuật toán policy-gradient ổn định, ít phân kỳ off-policy, phù hợp với tập dữ liệu nhỏ. Thư viện Stable-Baselines3 [Raffin et al., 2021] và Gymnasium [Towers et al., 2023] cung cấp hiện thực thực tế; phiên bản `MaskablePPO` cho phép *action masking* — chặn trước các hành động không hợp lệ theo giao thức, giải quyết vấn đề khám phá trong không gian hành động lớn.

2.4 Khoảng trống mà luận văn hướng tới

Phần lớn công trình trước dùng *mô hình xấp xỉ* của IDS để chấm điểm, hoặc chạy offline, hoặc không tính chi phí biến đổi. Chúng tôi lấp khoảng trống bằng cách: (i) chấm bằng *IDS thật* cho kết quả chính; (ii) luôn kèm *control âm tính* để chứng minh né tránh là thật; và (iii) báo cáo cả *kết quả phủ định* khi RL không tạo thêm giá trị — điều mà nhiều báo cáo né tránh không nêu.

Chương 3

Thiết kế hệ thống & Mô hình mối đe dọa

3.1 Mô hình mối đe dọa

3.1.1 Kẻ tấn công (red team)

Kẻ tấn công là người vận hành botnet, nắm được nội dung gói tin C2 dạng plaintext (PDU lệnh C2). Nó biết *protocol framing* của luồng mình (HTTP hay chuỗi bản ghi nhị phân) nhưng **không** biết nội dung bộ luật của defender. Nó chỉ được sửa **payload** (dữ liệu TCP/UDP), không được sửa metadata (IP, cổng).

3.1.2 Defender (blue team)

Defender là *Snort* 2.9.20 với bộ luật ET Open C2 (21 374 luật), chạy tĩnh, không phản ứng thích ứng. Mọi phán quyết (alert / không alert) được đọc trực tiếp từ nhị phân Snort.

3.1.3 Ràng buộc

- **Ngân sách hành động:** mỗi luồng chọn *một* cơ chế biến đổi từ 14 cơ chế; một số cơ chế chỉ áp dụng được cho một framing nhất định.
- **Giữ ngữ nghĩa:** phép biến đổi phải để lệnh C2 tới endpoint nguyên vẹn (định nghĩa ở mục 3.2.5).
- **Mục tiêu:** tối đa số luồng né được, tối thiểu chi phí biến đổi.

3.2 Môi trường `HiddenDefenderEnv`

Môi trường được hiện thực trong `ai_agent/hidden_defender_env.py`. Một *episode* = một luồng = một quyết định.

3.2.1 Quan sát (observation)

Agent chỉ thấy 8 đặc trưng thống kê (black-box), **không** thấy bộ luật:

Chỉ số	Đặc trưng
0	`is_udp` (luồng UDP hay không)
1	`n_pkts_norm` (số gói, chuẩn hoá)
2	`tot_bytes_norm` (tổng byte, chuẩn hoá)
3	`n_payload_pkts_norm` (số gói có payload)
4	`payload_mean_norm` (trung bình kích thước payload)
5	`payload_std_norm` (độ lệch chuẩn payload)
6	`payload_max_norm` (payload lớn nhất)
7	`payload_min_norm` (payload nhỏ nhất)

3.2.2 Không gian hành động

14 cơ chế (`ACTION_NAMES`), chỉ giữ lại những cơ chế đã được *đo* là hoặc né được hợp lệ, hoặc là đối chứng âm hữu ích:

Chỉ số	Cơ chế	Chi phí (`ACTION_COSTS`)
0	`split8`	0,60
1	`split16`	0,50
2	`split24`	0,40
3	`prepend4`	0,20
4	`prepend8`	0,20
5	`prepend12`	0,20
6	`pad16`	0,10
7	`reorder`	0,35
8	`ttl`	0,10
9	`corrupt8`	0,40
10	`overlap8`	0,35
11	`noop`	0,00
12	`http_header_pad`	0,05
13	`length_wrapper`	0,25

`split8/16/24`, `prepend4/8/12` là các cơ chế phân mảnh/chèn đã được đo; `pad16`, `reorder`, `ttl` được đo là 0/80 né (giữ làm đối chứng âm có thông tin); `corrupt8` cố ý phá ngữ nghĩa (để hình phạt -10 quan sát được); `noop` là hành động rỗng; `http_header_pad` và `length_wrapper` là hai cơ chế *nhận biết giao thức* (gọi là A và B).

3.2.3 Action masking (lọc hành động khả thi)

Hàm `mech_applicable()` chặn các cơ chế mà framing của luồng không thể mang: `http_header_pad` chỉ áp dụng cho framing `http`; `length_wrapper` chỉ cho framing `length2`. Các cơ chế không áp dụng được **bị loại khỏi mẫu số**, để mọi tỷ lệ phần trăm đều tính trên các cơ chế *có thể* tác động lên luồng đó. Điểm quan trọng: mask là một *bộ lọc tính khả thi*, không phải một phép biến đổi — `MaskablePPO` đặt logit của hành động bị mask về $-\infty$ trước softmax, nên hành động không khả thi không bao giờ được lấy mẫu.

3.2.4 Phần thưởng (reward)

Tình huống	Phần thưởng
Né được **và** ngữ nghĩa còn nguyên	+10 (`REWARD_EVADE`)
Né được **nhưng** phá ngữ nghĩa	−10 (`REWARD_BREAK_SEMANTICS`)
Vẫn bị báo động	−1 (`REWARD_ALERTED`)

Cộng thêm một phần thưởng nhỏ `PROTOCOL_MATCH_BONUS` = 0,15 *chỉ khi* một biến đổi đã né hợp lệ bằng `http_header_pad` trên luồng HTTP (nên nó không thể "chế ra" né tránh hay che một cơ chế đang thất bại), và trừ `ACTION_COSTS` theo cơ chế. Phần thưởng dương quan sát được trong kết quả chính là 9,6 (với `split24`) và 9,4 (với `split8`) — đúng bằng 10 trừ chi phí tương ứng (0,4 và 0,6).

3.2.5 Định nghĩa "giữ ngữ nghĩa"

Hàm `semantics_intact()` yêu cầu lệnh C2 phải tới endpoint nguyên vẹn. Vì không mô hình hoá được ngữ pháp của mọi giao thức, chúng tôi dùng mức chặt:

- Nếu số gói thay đổi (phân mảnh), so sánh *luồng byte được ghép lại*: lệnh parse được từ luồng gốc phải bằng lệnh parse được từ luồng biến đổi.
- Nếu số gói giữ nguyên, với *mọi* gói mà payload gốc sinh ra một lệnh, lệnh đó phải bằng nhau trước và sau.

Đây là mức "byte-equality" chặt. Một mức lỏng hơn (chỉ kiểm tra chuỗi con) từng khiến mọi cơ chế đều được tính là né (xem lỗi số 3, Chương 6).

Chương 4

Phương pháp

4.1 Dữ liệu và phân chia tập

4.1.1 Nguồn dữ liệu

Corpus lấy từ các capture malware thật của Stratosphere IPS [Lab and in Prague, 2013]. Hai capture được dùng cho tập huấn luyện và test:

- `botnet-capture-20110811-neris` (Stratosphere MCFP)
- `botnet-capture-20110819-bot` (CTU-13)

Cả hai capture đều *có mặt trong cả train và test*, để một phương pháp không thể "thắng" bằng cách nhận diện "đây là capture nào".

4.1.2 Quy tắc chọn và phân chia

Chỉ giữ lại các luồng mà Snort thật *thật sự* phát hiện (để có cái mà né). Phân chia cố định bởi `corpus_config.py`:

Tham số	Giá trị
`n_train`	1 153
`n_test`	40
`SEED`	42
`OBS_DIM`	8

Hàm `split_flows()` dùng `random.Random(42)` để shuffle rồi lấy 40 luồng đầu làm test — phân chia *xác định* (deterministic) để tái lập được. Test cố ý giữ ở 40 luồng để mọi eval/control trước đó vẫn so sánh được.

4.2 Huấn luyện

4.2.1 Thuật toán

Dùng `MaskablePPO` [Schulman et al., 2017, Raffin et al., 2021] với policy `MlpPolicy`. Vì mỗi luồng là một quyết định độc lập nhưng PPO cần trajectory, một wrapper mỏng (`BatchEnv`) gộp n luồng liên tiếp thành *một* episode dài, để rollout là n bước quyết định thật thay vì n episode độ dài 1.

Siêu tham số	Giá trị
`n_steps`	$= n_flows$ (mặc định 64)
`batch_size`	$= n_flows$
`n_epochs`	10
`learning_rate`	3×10^{-4}
`ent_coef`	0,0151
`gamma`	0,0 (mỗi luồng độc lập)
`seed`	42
Ngưỡng dừng (`TARGET`)	0,90

`gamma`=0 vì mỗi bước là một quyết định độc lập, không có phần thưởng tương lai.

4.2.2 Hai bộ chấm điểm

Đây là điểm *quan trọng* về tính trung thực:

- **Surrogate nhanh** (`RealRulesReplica`): parse bộ luật ET Open thật và tái hiện các sid thật sự kích hoạt; dùng để *huấn luyện* cho nhanh (∼0,33 ms/luồng). Nó **bỏ sót 20,6%** luật thật (các luật dùng `pcre`, `byte_test`, `byte_extract`, `byte_jump`).
- **Snort thật** (`-real-snort`): chấm mọi hành động bằng nhị phân thật (∼1,3 s/luồng); dùng cho *kết quả chính*.

Việc tách hai bộ chấm là có chủ đích: con số 99,2% khi huấn luyện bằng surrogate *không phải* cùng đại lượng với con số 90,6% khi huấn luyện bằng Snort thật.

4.3 Quy trình đo lường

4.3.1 Một cặp (flow, cơ chế) cho mỗi pcap

Gộp 32 cặp vào một pcap từng khiến Snort phát 4 alert trong khi chấm riêng lẻ tìm được 12 phát hiện — lưu lượng trộn lẫn làm giảm alert trên toàn capture. Vì vậy ma trận cơ chế chạy với `batch_size`=1: 224 lần gọi Snort, ∼80 s.

4.3.2 Control âm tính (bắt buộc)

Mỗi lần đo đều kèm một control: các luồng *chưa* biến đổi phải bị Snort báo động. Nếu control ngừng báo động, mọi con số khác bị bỏ. Control này là thứ đã bắt được hai lỗi tệ nhất của dự án.

4.3.3 Trần oracle và các baseline

Để con số "PPO đạt 100%" có ý nghĩa, cần một trần:

Phương pháp	Bản chất	Vai trò
`random_sweep`	thứ tự cơ chế ngẫu nhiên, thử tới khi hợp lệ	sàn
`greedy`	dùng lại cơ chế hiệu quả trên luồng gần nhất	heuristic mạnh
`supervised_tree`	cây quyết định trên nhãn oracle	**trần**
`ppo`/`maskable_ppo`	policy đã huấn luyện	đối tượng cần đánh giá

4.3.4 Verifier tự động

`controls/verify_report_claims.py` tái suy diễn mọi con số trong báo cáo từ file JSON gốc và *thất bại* nếu có sai lệch. Verifier kiểm tra các *bất biến* (*e.g.* `queries == epochs * n_flows`) chứ không ghim hằng số, vì dừng sớm làm hằng số thay đổi mỗi lần chạy. Bộ test (`tests/`) gồm 69 test đều pass.

Chương 5

Kết quả & Đánh giá

Mọi con số trong chương này được đọc từ file JSON trong `snort_validation/reports/`. Không có con số nào được ước lượng.

5.1 Kết quả chính (Snort thật)

Từ `ppo_masked_test.json` — `eval_masked_ppo_test.py -real-snort`, trên 40 luồng held-out:

Chỉ số	Giá trị
method	`maskable_ppo`
scorer	`real_snort`
split	test
n_flows	40
alerted	**0**
valid_evasion	**40 / 40 (100,0%)**
`masked_actions_used`	0
Control âm tính (`control_noop_real_snort.json`)	**40 / 40 bị báo động**

Policy chỉ dùng đúng hai cơ chế:

Cơ chế	Số luồng
`split8`	24 (toàn bộ luồng UDP/neris)
`split24`	16 (toàn bộ luồng HTTP/TCP)

Mọi luồng đều có `semantics_ok = true`; phần thưởng là 9,4 (split8) và 9,6 (split24). Việc phân bố framing/action là hoàn hảo (24 neris→split8, 16 http→split24), đúng như kỳ vọng về masking nhận biết giao thức.

5.1.1 Control âm tính

`control_noop_real_snort.json`: 40 luồng *không* biến đổi, tất cả `alert = true`, `evaded = false`. Tỷ lệ né của control là **0%**. Đây là bằng chứng định lượng rằng 40/40 của agent là thật, không phải do Snort không chạy.

5.2 Số liệu huấn luyện

Lần chạy	Bộ chấm	flows	né được	queries	epochs	file
Surrogate	`RealRulesReplica`	128	127 (99,2%)	896	7	`ppo_hidden_defender.jso`
Snort thật	nhị phân thật	64	58 (90,6%)	640	10	`ppo_realsnort_train.jso`

Cả hai lần đều dừng ở ngưỡng $\geq 90\%$, không phải ở hội tụ. Không lần nào là trần thật.

5.3 Ma trận cơ chế (Snort thật, 8 luồng HTTP)

Từ `hidden_defender_matrix_test.json` — 8 luồng HTTP, 112 ô, chấm một cặp mỗi pcap:

Cơ chế	Né được	Né được **và** ngữ nghĩa nguyên
`split8`	8/8	**8/8**
`split16`	8/8	**8/8**
`split24`	8/8	**8/8**
`http_header_pad`	8/8	**8/8**
`prepend4`	8/8	0/8
`prepend8`	8/8	0/8
`prepend12`	8/8	0/8
`overlap8`	8/8	0/8
`length_wrapper`	8/8	0/8
`corrupt8`	1/8	0/8
các cơ chế còn lại	0/8	0/8

Đây là bảng quan trọng nhất vì cột phải là nơi công việc trung thực thể hiện: **sáu cơ chế đạt 8/8 né nhưng 0/8 né hợp lệ**. Chúng không đánh bại IDS — chúng phá kênh C2, và một hàm thưởng ngây thơ sẽ cho chúng điểm tuyệt đối. `length_wrapper` còn là code chết: 0/8 hợp lệ *và* 0/16 áp dụng được trên luồng neris.

5.4 RL có thật sự cần thiết không? (phát hiện làm phức tạp câu chuyện)

Từ `hidden_defender_real_snort_test.json` — 8 luồng HTTP, Snort thật:

Phương pháp	Queries	Né được
`random_sweep`	25	8/8
`greedy`	13	8/8
`supervised_tree`	8	8/8
ppo	8	8/8

Trên tập huấn luyện 64 luồng, so sánh còn rõ hơn: `greedy` né 58/64 với **149** queries, trong khi PPO cũng né 58/64 nhưng tốn **704** queries — gấp **4,7 lần**. Và trên tập HTTP, bốn cơ chế `split8`/`split16`/ `split24`/`http_header_pad` đều giải được 8/8, nên hành động tối ưu gần như *hằng số theo framing*, không phải theo luồng.

Cách đặt vấn đề trung thực cho báo cáo: đây *không phải* "RL đánh bại các baseline". Mà là: "môi trường là thật, phép đo là vững, và dưới *defender này* bài toán tìm kiếm quá dễ để RL tạo thêm giá trị — hành động thắng gần như cố định." Đây là một *kết quả phủ định về cách đóng khung bài toán*, không phải về agent, và nó bảo vệ được.

5.5 Scale-up (Snort thật, neris-20110811, 8 vòng)

Từ `final_scale_up_summary.json`:

Số luồng	Né được	Tỷ lệ né	mean corrupt
24	24	100,0%	0,00
100	94	94,0%	8,69
300	287	95,7%	7,71
323	69	**21,4%**	0,00

Kết quả **không đơn điệu**: từ 24 lên 300 luồng vẫn cao, rồi sụp ở 323 luồng. Chúng tôi nêu thẳng hiện tượng này thay vì làm mượt nó (xem Chương 6).

5.6 Kiểm tra chéo capture (cross-capture)

Từ `cross_capture_summary.json` (cost = 0,6), sinh lại từ các báo cáo per-capture:

Capture	Baseline bị bắt	Policy né	Corrupt-all né	Kết luận
botnet-capture-20110810-neris	23/24	4	4	hoà
botnet-capture-20110811-neris	24/24	24	24	hoà
capture-win13	24/24	0	0	hoà

Trung bình 38,89%, độ lệch chuẩn 43,74%, min 0%, max 100%. `vs_control_counts = {tie: 3}`.

Cột "Corrupt-all né" là điểm mấu chốt: trên cả ba capture, policy né đúng bằng số luồng mà hành động "làm hỏng mọi gói có thể" cũng né được, với cùng mức biến đổi trung bình. **Policy hoà control 3/3** — nó không học được gì mà hành động tầm thường "corrupt-everything" chưa đạt được. Vì vậy con số 100% trên capture 0811 là *tính chất của control đủ mạnh*, không phải bằng chứng cho RL agent. `capture-win13` ở 0% là tín hiệu trung thực rằng policy không tổng quát hoá qua capture.

5.7 Kiểm định surrogate so với Snort thật

Từ `surrogate_sweep_14mech.json` — 14 cơ chế × 16 luồng, 224 lần gọi Snort thật, 2 359 s:

Chỉ số	Giá trị
Đồng thuận tổng thể	161/224 (71,9%)
Đồng thuận trên ô áp dụng được	103/116 (88,8%)
surrogate false-safe (báo né khi Snort báo động)	**0**

`surrogate_vs_real_snort.json`: 8/8 đồng thuận, 0 false-safe. Con số "0 false-safe" là con số quan trọng nhất: một false-safe nghĩa là surrogate nói dối agent rằng nó đã né — đúng chế độ thất bại có thể vô hiệu hoá mọi thí nghiệm chấm bằng surrogate. Nó *không* xảy ra ở đây.

5.8 Sweep chi phí corrupt

Từ `sweep_corrupt_cost.json`: né 100% từ cost 0,2 đến 2,0, rồi 0% ở cost 4,0; mean corrupt bị ghim ở 2,88 qua mọi mức cost. Tham số cost không định hình policy mà chỉ bật/tắt nó.

Chương 6

Thảo luận

6.1 Vì sao `MaskablePPO` hợp ở đây

- **Action masking giải quyết khám phá.** Với framing, cơ chế khả thi bị lọc trước, nên agent chỉ cân nhắc các biến đổi hợp lệ theo giao thức.
- **`PPO` ổn định trên corpus nhỏ.** `PPO` tránh phân kỳ off-policy, phù hợp khi dữ liệu ít (1 153 luồng).
- **Quan sát black-box 8 chiều.** Agent không thấy bộ luật, chỉ thấy thống kê luồng — đúng như một kẻ tấn công thật.

6.2 Chi phí và ý nghĩa

Mỗi biến đổi có chi phí (`ACTION_COSTS`): phân mảnh tốn băng thông/CPU, chèn header HTTP rẻ và tự nhiên. Trong phiên bản bandit đã nghỉ hưu, chi phí được hình thức hoá qua `break_even_cost` $\approx 3{,}2$ gói: với N gói có thể sửa trong một luồng, "làm hỏng mọi gói" chỉ hoà vốn khi chi phí vượt `EVASION_BONUS`$/N$. Với môi trường hiện tại, chi phí là các hằng số cố định theo cơ chế; điều đáng chú ý là sweep chi phí *không* định hình policy (mean corrupt ghim ở 2,88) — đây là một dấu hiệu overfit cần nêu thẳng.

6.3 Vì sao các con số nên được tin: những lỗi đã bắt được

Mỗi lỗi sau từng tạo ra kết quả sai trông rất hợp lý, và bị bắt bởi control hoặc kiểm tra chéo — không phải bởi đọc code:

#	Lỗi	Cách phát hiện
1	Trích xuất luồng một chiều (mất gói ngược)	baseline phát hiện 0/24 (bất khả)
2	Viết lại gói tin ngây thơ (sửa cả phản hồi server)	vẫn 0/24 sau khi sửa lỗi 1
3	`semantics_intact` chấp nhận chứa chuỗi con	mọi cơ chế đạt 100%
4	Chấm điểm theo lô làm triệt alert	pcap 32 cặp cho 4 alert, chấm riêng 12
5	Luật ET throttle `by_src`	địa chỉ query tái dùng alert một lần rồi im
6	Cửa sổ đọc alert ngắn hơn nhịp flush của Snort	`-resident` báo baseline 0/24

Lỗi số 3 là lỗi cần trình bày cẩn thận nhất. Nó là khác biệt giữa "agent né được IDS" và "agent né được IDS *trong khi vẫn giữ malware hoạt động*". Kết quả 100% trước đó đã bị thu hồi vì nó. Mức hiện tại là byte-equality.

6.4 Hạn chế

- **Chỉ một seed, một cặp capture.** Kết quả chính là một seed trên tập test 40 luồng, tất cả từ hai capture cùng họ. Chưa có trần hội tụ đa-seed.
- **Defender tĩnh, đơn luật.** Chỉ bộ luật ET Open tĩnh; chưa thử IDS thích ứng hay phát hiện bất thường.
- **Surrogate bỏ sót 20,6% luật.** Trên corpus này không ảnh hưởng (chỉ 3 sid kích hoạt mỗi capture) nhưng sẽ ảnh hưởng trên lưu lượng khác. Đồng thuận `split8` chỉ 5/16 (31,2%) — surrogate tệ nhất đúng nơi policy hoạt động.
- **Không tổng quát hoá chéo capture.** win13 ở 0%. Đây là ngõ cụt của *không gian hành động*: các luồng win13 gần như không có payload (0/97 gói có `Raw`), nên không byte nào sửa được; Snort vẫn alert 24/24 vì luật establishment/dsize không cần payload. Kết luận đúng là phụ thuộc *trường luật*, không phải phụ thuộc họ botnet.
- **Scale không đơn điệu.** Sụp ở 323 luồng chưa được giải thích; cần chẩn đoán hoặc nói thẳng "chúng tôi chưa biết".
- **Cả hai cơ chế thắng đều là phân mảnh** (`split8`, `split24`) — policy chọn cách né rẻ nhất còn giữ ngữ nghĩa, không phải một kho cơ chế đa dạng.

6.5 Các bất nhất cần giải quyết

Tài liệu `docs/REPORT_WRITING_GUIDE.md` và `docs/SIC_AI_FRAMEWORK_VALIDATION.md` còn mang một số con số *đã lỗi thời*: ví dụ mean evasion 73,3%, control 71/71, train 1 113 luồng, không gian hành động "65 536", phần thưởng $+100/-2/-50$. Các con số này **không** khớp với file JSON gốc. Luận văn này dùng *file JSON gốc* làm nguồn chân lý (xem Phụ lục E). Một số tài liệu khác (`all_models_comparison.json`, `mechanism_matrix.json`) thuộc thế hệ hệ thống cũ và không so sánh được với kết quả hiện tại.

6.6 Hướng phát triển

- **Defender dị thể/thích ứng:** cấp cho mỗi luồng một luật C2 với `depth`/`offset` khác nhau, để offset phân mảnh thắng thay đổi thật sự theo luồng — khi đó RL mới có việc để làm.
- **Đa seed và kiểm tra chéo họ botnet thật** (không chỉ cùng nhị phân).
- **Retrain surrogate theo kiểu DAgger** (gán nhãn chính biến đổi của agent mới bằng Snort thật rồi huấn luyện lại) để đóng khoảng cách surrogate–thực tế.
- **Mở rộng không gian hành động** sang metadata/TTL/phân mảnh để không bị ngõ cụt trên các luồng không payload như win13.

Chương 7

Kết luận

Luận văn cho thấy một agent `MaskablePPO` có thể biến đổi gói tin C2 thật để né *Snort* 2.9.20 (ET Open) một cách đáng tin cậy: **40/40 luồng held-out né hợp lệ dưới Snort thật**, trong khi control âm tính bị báo động 40/40. Mọi hành động đều được chấm bằng nhị phân thật, và một verifier tự động 81 kiểm tra bảo đảm không con số nào trôi dạt khỏi nguồn.

Nhưng luận văn cũng kết luận trung thực rằng, *dưới defender tĩnh này*, RL chưa tạo thêm giá trị: bài toán tìm kiếm gần như suy biến (một hành động gần như cố định mỗi framing), nên PPO không vượt greedy; và trên kiểm tra chéo capture (trung bình 38,89%, std 43,74%), policy *hoà* control "corrupt-everything" trên cả ba capture — con số né chéo capture không phải thành tựu của RL. Một capture đạt 0% vì lý do không gian hành động, không phải vì họ botnet.

Giá trị của công trình nằm ở *sự trung thực của phép đo*: control âm tính, trần oracle, một cặp mỗi pcap, verifier tự động, và việc báo cáo thẳng cả kết quả phủ định. Con đường phía trước rõ ràng: làm defender dị thể/thích ứng để bài toán thật sự cần đến RL, rồi kiểm tra đa seed và chéo họ botnet.

Chương 8

Phụ lục

8.1 A. Thống kê corpus

Chỉ số	Giá trị
Tổng luồng train	1 153
Tổng luồng test	40
`obs_dim`	8
Capture train	`20110811-neris`, `20110819-bot`
Capture test	`20110811-neris`, `20110819-bot`
Framing trong tập test	24 `neris` (UDP) + 16 `http` (TCP)

8.2 B. Chi tiết hiện thực

Snort: 2.9.20, bộ luật ET Open C2 (21 374 luật), lệnh `snort -c <conf> -r <pcap> -l <dir> -q -A fast`; phán quyết đọc từ file fast-alert. Chấm một cặp (flow, cơ chế) mỗi pcap. **Thời gian:** eval chính `-real-snort` ∼172 s, control ∼166 s, ma trận cơ chế ∼11,2 s/ô (112 ô ≈20 phút). **Hyperparameters:** xem Chương 4. **Verifier:** 81/81 kiểm tra pass. **Test:** 69 test pass.

8.3 C. Bảng kết quả per-flow (40 luồng test)

Bảng 8.1 liệt kê toàn bộ 40 luồng held-out: framing, cơ chế được chọn, alert, ngữ nghĩa, phần thưởng. Tất cả đều `alert=false`, `evaded=true`, `semantics_ok=true`.

8.4 D. Control âm tính và kiểm tra ngữ nghĩa

Control `control_noop_real_snort.json`: 40 luồng, tất cả bị báo động (`alerted=40`, `n=40`), tỷ lệ né 0%. Mức ngữ nghĩa dùng trong môi trường là byte-equality / so khớp lệnh sau khi ghép luồng (mục 3.2.5). Chỉ khi `evaded AND semantics_ok` thì một luồng mới được tính là né hợp lệ (`QueryCounter.valid()`).

8.5 E. Ghi chú chỉnh sửa các con số lỗi thời

Một số tài liệu nội bộ từng mang con số *đã bị vượt qua*. Bảng dưới đối chiếu con số trong tài liệu với con số trong file JSON gốc (nguồn chân lý):

Chỉ số	Giá trị cũ (tài liệu)	Giá trị đúng (JSON gốc)
Mean evasion chéo capture	73,3% / 62,5%	**38,89%**
Std chéo capture	10,8% / 44,49%	**43,74%**
Control	71/71 bị bắt	**40/40 bị bắt**
Tập test né được	29/40	**40/40**
Luồng train	1 113	**1 153**
Không gian hành động	65 536	**14 cơ chế**
Phần thưởng	$+100/-2/-50$	$+10/-10/-1$

Nguyên nhân gốc của dữ liệu tổng hợp lỗi thời: script tổng hợp cũ lọc input theo chuỗi tên file `"summary"` và bỏ sót khi cập nhật, để lại một file cũ. Hiện tại script chọn input theo *schema* và verifier khẳng định file tổng hợp khớp với mọi báo cáo per-capture, nên một file tổng hợp lỗi thời sẽ làm verifier *thất bại* thay vì được công bố.

Bảng 8.1: Kết quả per-flow trên tập test held-out (Snort thật).

#	Capture	Framing	Action	Reward	Sem.
0	20110811-neris	http	split24	9,6	✓
1	20110819-bot	neris	split8	9,4	✓
2	20110811-neris	http	split24	9,6	✓
3	20110819-bot	neris	split8	9,4	✓
4	20110819-bot	neris	split8	9,4	✓
5	20110819-bot	neris	split8	9,4	✓
6	20110819-bot	neris	split8	9,4	✓
7	20110819-bot	neris	split8	9,4	✓
8	20110819-bot	neris	split8	9,4	✓
9	20110819-bot	neris	split8	9,4	✓
10	20110819-bot	neris	split8	9,4	✓
11	20110811-neris	http	split24	9,6	✓
12	20110819-bot	neris	split8	9,4	✓
13	20110819-bot	neris	split8	9,4	✓
14	20110811-neris	http	split24	9,6	✓
15	20110819-bot	neris	split8	9,4	✓
16	20110819-bot	neris	split8	9,4	✓
17	20110811-neris	http	split24	9,6	✓
18	20110819-bot	neris	split8	9,4	✓
19	20110811-neris	http	split24	9,6	✓
20	20110819-bot	neris	split8	9,4	✓
21	20110811-neris	http	split24	9,6	✓
22	20110811-neris	http	split24	9,6	✓
23	20110811-neris	http	split24	9,6	✓
24	20110811-neris	http	split24	9,6	✓
25	20110811-neris	http	split24	9,6	✓
26	20110819-bot	neris	split8	9,4	✓
27	20110819-bot	neris	split8	9,4	✓
28	20110819-bot	neris	split8	9,4	✓
29	20110819-bot	neris	split8	9,4	✓
30	20110811-neris	http	split24	9,6	✓
31	20110811-neris	http	split24	9,6	✓
32	20110819-bot	neris	split8	9,4	✓
33	20110811-neris	http	split24	9,6	✓
34	20110819-bot	neris	split8	9,4	✓
35	20110811-neris	http	split24	9,6	✓
36	20110819-bot	neris	split8	9,4	✓
37	20110819-bot	neris	split8	9,4	✓
38	20110811-neris	http	split24	9,6	✓
39	20110819-bot	neris	split8	9,4	✓

Tài liệu tham khảo

Battista Biggio and Fabio Roli. Evasion attacks against machine learning classifiers. In *2013 10th IEEE International Conference on Machine Learning and Applications*, pages 345–350. IEEE, 2013.

Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 39–57. IEEE, 2016.

Luca Demetrio et al. Intriguing properties of adversarial ml attacks in the problem space. *arXiv preprint arXiv:1907.07932*, 2020.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

Stratosphere IPS Lab and CTU in Prague. Ctu-13: Botnet traffic capture. Stratosphere IPS dataset distribution, 2013. URL https://www.stratosphereips.org/datasets-ctu13.

Nicolas Papernot, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z Berkay Celik, and Ananthram Swami. The limitations of deep learning in adversarial settings. *arXiv preprint arXiv:1511.04508*, 2015.

Fabio Pierazzi, Nicholas Carlini, et al. Intriguing properties of adversarial examples for malware detection. *IEEE Transactions on Information Forensics and Security*, 2019.

Proofpoint, Inc. Emerging threats: Open source intrusion detection rules. Emerging Threats Open ruleset distribution, 2023. URL https://rules.emergingthreats.net.

Martin L Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

Antonin Raffin, Ashley Hill, Matthias Ernestus, and Michael Gleicher. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

Martin Roesch. Snort: An open source network intrusion detection system, 2023. URL https://www.snort.org.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Niyaz R Silva et al. Ids evasion via network manipulation: A machine learning perspective. In *2017 IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 799–805. IEEE, 2017.

Mark Towers et al. Gymnasium: A standard api for reinforcement learning, 2023. URL https://gymnasium.farama.org.